Strategy Note: MFQ Test Plan - E-Commerce System

How priority was assigned

Priorities are assigned by a fixed rule. Two questions make a case P0: **could a customer notice the defect before we do**, and **can we undo it**. Inventory deduct, refund and reservation release fail both, which is why they hold the highest density of P0 across the 81 cases (47 P0 / 31 P1 / 3 P2).

- **P0 - 47 cases:** money, inventory truth, or a one-way door. Run every build and every candidate.
- **P1 - 31 cases:** the normal customer journey and its contract regressions. Run every test round.
- **P2 - 3 cases:** robustness and hygiene. Triage at milestones.

Models are used as test oracles, not as documentation

Each M layer model hands the suite something falsifiable. The **state machine** hands us guards, which become "the illegal transition must be refused" (F3-16, F4-11) - the check that stops a refund being issued against an already-shipped order. The **inventory invariant** `available + reserved + frozen = on_hand − safety_stock` is asserted before and after every inventory case and sampled continuously under load. The **consistency table** turns an unverifiable phrase into a testable one: every eventual boundary has a named reconciliation point with a numeric tolerance, and a boundary without one is treated as an untested bug. That is why the M layer carries five models and the dedicated sections carry three specifications: the 81 executable cases live in F and Q, and not one of them checks "the order list renders" - each asserts something a model declares.

Trade-offs made on purpose

Decision	Cost	Why it is still right
Depth on inventory sync, breadth on payments	Only the primary channel gets a full matrix	Channel two shares the callback contract covered by F4-07; its own matrix is deferred with a named approver
A mock WMS with fault injection, not the real WMS	The mock can drift from the real contract	Seven fault classes become deterministic on demand; a real WMS cannot be made to lose an ACK. Contract tests gate every mock change
API-first, one UI smoke journey	Checkout UI regression is not covered	Inventory and money change at the API; the gap is a named residual risk, not an oversight
8-hour soak instead of 72 hours	Very slow leaks could survive	8h at 1.2x peak plus hourly reconciliation exposes drift growth; the extra 64 hours buy little for a release slot
No "exactly once" assertions	No exactly-once claim	The design assumes at-least-once everywhere; testing a property we refuse to rely on validates the wrong thing
Observability cases at P0/P1 weight	Execution time up front	An alert that is never verified is an outage with extra latency

Not tested, and recorded as residual risk

Payment-gateway internals, carrier routing, coupon rules, full UI regression and WMS internal logic

- each is a third party or a lower-value surface this iteration, and each carries a named approver.

If the time box changes

- **Half the time:** keep every P0, drop the P2 set, run the F2 fault matrix once instead of twenty times. Never drop the invariant assertions - cheapest coverage per second in the plan.
- **Double the time:** add the second payment channel, a multi-region failover drill, and 72h of message-layer fuzzing with automatic drift detection.

The numbers that gate the release

No oversell. Zero negative inventory buckets. Inventory diff ratio under 0.05% per warehouse after 24 hours of continuous reconciliation. Zero unexplained differences. Every seeded drift repaired within SLA by a named compensation job. These five checks, rather than a pass-rate percentage, decide the release.